Name of Student: Husam Nour	**PRN:25070122086**
Semester: III	**Year : AY 2026-27**
Subject: Programming with Java	
ASSIGNMENT No : 24	
TITLE : JDBC with GUI	

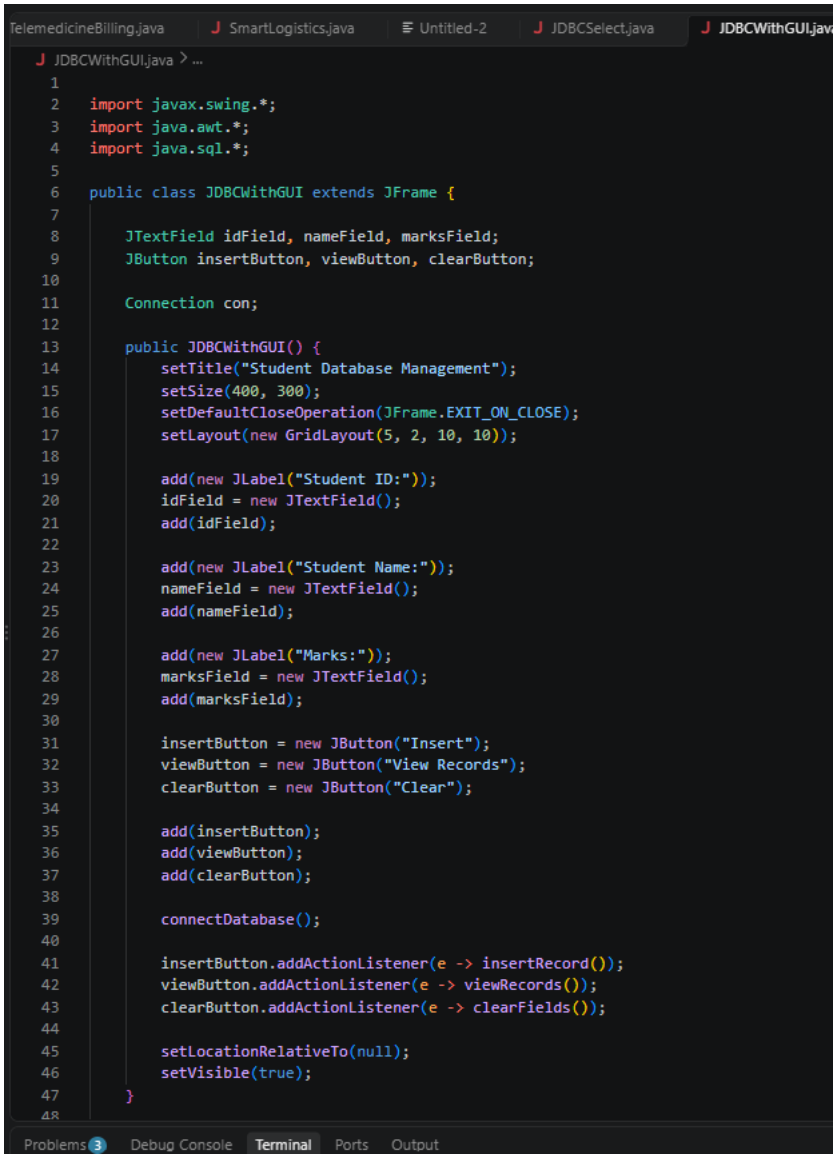
Problem Statement

Develop a Java GUI application integrated with JDBC to perform database operations."

Learning Objectives

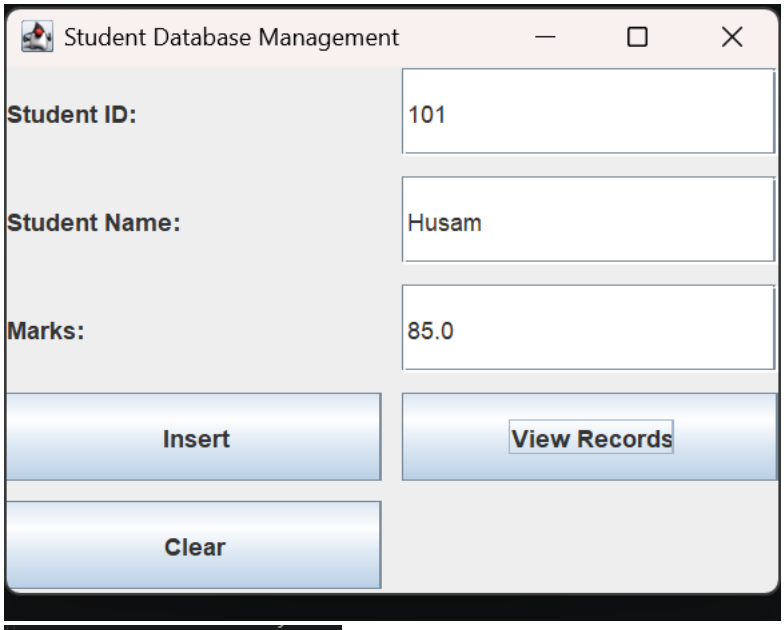
To integrate Java GUI applications with databases using JDBC.

Programme


TelemedicineBilling.java
SmartLogistics.java
Untitled-2
JDBCSelect.java
JDBCWithGUI.java
JDBCWithGUI.java > ...
1
2 import javax.swing.*;
3 import java.awt.*;
4 import java.sql.*;
5
6 public class JDBCWithGUI extends JFrame {
7
8 JTextField idField, nameField, marksField;
9 JButton insertButton, viewButton, clearButton;
10
11 Connection con;
12
13 public JDBCWithGUI() {
14 setTitle("Student Database Management");
15 setSize(400, 300);
16 setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
17 setLayout(new GridLayout(5, 2, 10, 10));
18
19 add(new JLabel("Student ID:"));
20 idField = new JTextField();
21 add(idField);
22
23 add(new JLabel("Student Name:"));
24 nameField = new JTextField();
25 add(nameField);
26
27 add(new JLabel("Marks:"));
28 marksField = new JTextField();
29 add(marksField);
30
31 insertButton = new JButton("Insert");
32 viewButton = new JButton("View Records");
33 clearButton = new JButton("Clear");
34
35 add(insertButton);
36 add(viewButton);
37 add(clearButton);
38
39 connectDatabase();
40
41 insertButton.addActionListener(e -> insertRecord());
42 viewButton.addActionListener(e -> viewRecords());
43 clearButton.addActionListener(e -> clearFields());
44
45 setLocationRelativeTo(null);
46 setVisible(true);
47 }
48
Problems 3
Debug Console
Terminal
Ports
Output

```
JDBCWithGUI.java > ...
  6   public class JDBCWithGUI extends JFrame {
 64       void insertRecord() {
 72
 73           ps.executeUpdate();
 74
 75           JOptionPane.showMessageDialog(this, "Record inserted successfully!");
 76           ps.close();
 77
 78       } catch (SQLException | NumberFormatException e) {
 79           JOptionPane.showMessageDialog(this, "Error: " + e.getMessage());
 80       }
 81   }
 82
 83   void viewRecords() {
 84       try {
 85           Statement stmt = con.createStatement();
 86           ResultSet rs = stmt.executeQuery("SELECT * FROM student");
 87
 88           StringBuilder records = new StringBuilder();
 89
 90           while (rs.next()) {
 91               records.append("ID: ").append(rs.getInt("id"))
 92                      .append(", Name: ").append(rs.getString("name"))
 93                      .append(", Marks: ").append(rs.getDouble("marks"))
 94                      .append("\n");
 95           }
 96
 97           JOptionPane.showMessageDialog(
 98               this,
 99               records.length() == 0 ? "No records found." : records.toString()
100           );
101
102           rs.close();
103           stmt.close();
104
105       } catch (SQLException e) {
106           JOptionPane.showMessageDialog(this, "Error: " + e.getMessage());
107       }
108   }
109
110   void clearFields() {
111       idField.setText("");
112       nameField.setText("");
113       marksField.setText("");
114   }
115
```

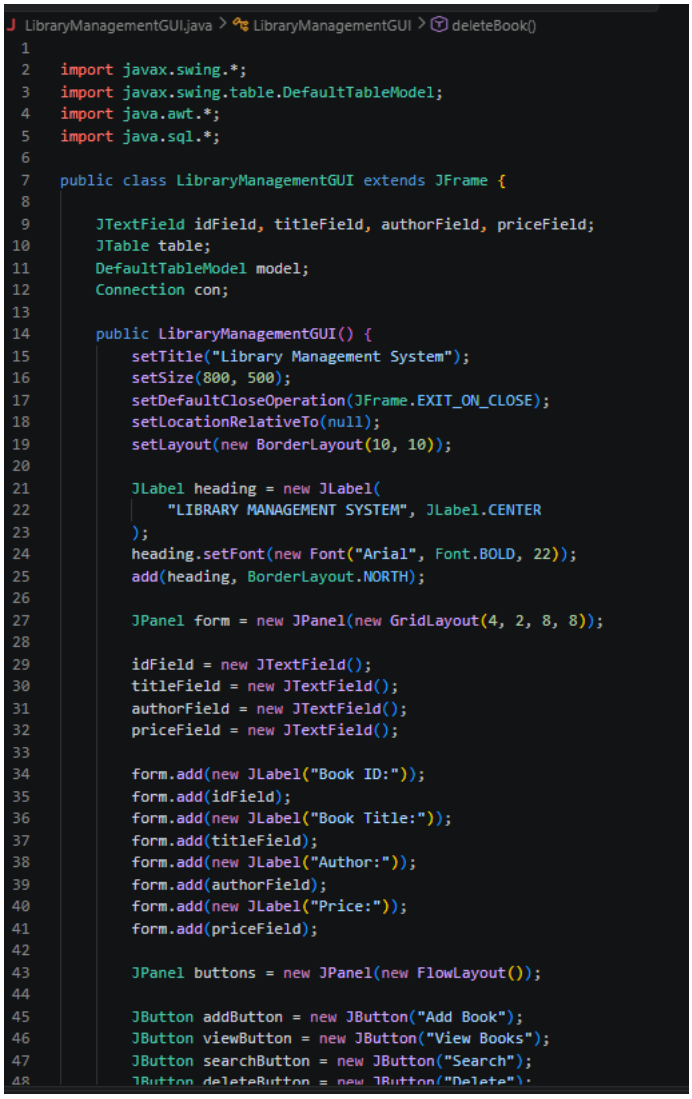
Output




```
PS C:\Users\hp\Documents\husamm> java
Database connected successfully.
```

Exercise 1. Create a Library Management System GUI connected with database

Code


LibraryManagementGUI.java > LibraryManagementGUI > deleteBook()
1
2 import javax.swing.*;
3 import javax.swing.table.DefaultTableModel;
4 import java.awt.*;
5 import java.sql.*;
6
7 public class LibraryManagementGUI extends JFrame {
8
9 JTextField idField, titleField, authorField, priceField;
10 JTable table;
11 DefaultTableModel model;
12 Connection con;
13
14 public LibraryManagementGUI() {
15 setTitle("Library Management System");
16 setSize(800, 500);
17 setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
18 setLocationRelativeTo(null);
19 setLayout(new BorderLayout(10, 10));
20
21 JLabel heading = new JLabel(
22 "LIBRARY MANAGEMENT SYSTEM", JLabel.CENTER
23);
24 heading.setFont(new Font("Arial", Font.BOLD, 22));
25 add(heading, BorderLayout.NORTH);
26
27 JPanel form = new JPanel(new GridLayout(4, 2, 8, 8));
28
29 idField = new JTextField();
30 titleField = new JTextField();
31 authorField = new JTextField();
32 priceField = new JTextField();
33
34 form.add(new JLabel("Book ID:"));
35 form.add(idField);
36 form.add(new JLabel("Book Title:"));
37 form.add(titleField);
38 form.add(new JLabel("Author:"));
39 form.add(authorField);
40 form.add(new JLabel("Price:"));
41 form.add(priceField);
42
43 JPanel buttons = new JPanel(new FlowLayout());
44
45 JButton addButton = new JButton("Add Book");
46 JButton viewButton = new JButton("View Books");
47 JButton searchButton = new JButton("Search");
48 JButton deleteButton = new JButton("Delete");

```java
public class LibraryManagementGUI extends JFrame {
    public LibraryManagementGUI() {

        JButton addButton = new JButton("Add Book");
        JButton viewButton = new JButton("View Books");
        JButton searchButton = new JButton("Search");
        JButton deleteButton = new JButton("Delete");
        JButton clearButton = new JButton("Clear");

        buttons.add(addButton);
        buttons.add(viewButton);
        buttons.add(searchButton);
        buttons.add(deleteButton);
        buttons.add(clearButton);

        JPanel topPanel = new JPanel(new BorderLayout(8, 8));
        topPanel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));
        topPanel.add(form, BorderLayout.CENTER);
        topPanel.add(buttons, BorderLayout.SOUTH);

        model = new DefaultTableModel(
            new String[]{"Book ID", "Title", "Author", "Price"}, 0
        );
        table = new JTable(model);

        add(topPanel, BorderLayout.CENTER);
        add(new JScrollPane(table), BorderLayout.SOUTH);

        connectDatabase();

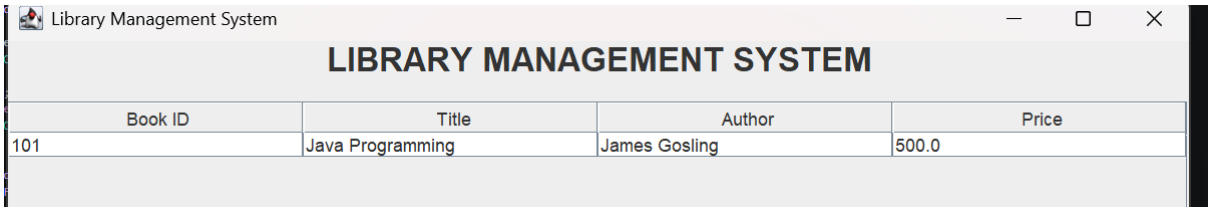
        addButton.addActionListener(e -> addBook());
        viewButton.addActionListener(e -> viewBooks());
        searchButton.addActionListener(e -> searchBook());
        deleteButton.addActionListener(e -> deleteBook());
        clearButton.addActionListener(e -> clearFields());

        viewBooks();
        setVisible(true);
    }

    void connectDatabase() {
        try {
            con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/college",
                "root",
                "Husam123"
            );
            System.out.println("Database connected successfully.");
```

Problems 3 | Debug Console | Terminal | Ports | Output

Output



2. Create a Book Issue Tracking System GUI application using Java Swing connected to a database. The system should store book ID, student name, issue date, and return date, and allow record management.

Code

```java
public class BookIssueTrackingGUI extends JFrame {
    public BookIssueTrackingGUI() {

        JButton addButton = new JButton("Add Record");
        JButton viewButton = new JButton("View Records");
        JButton searchButton = new JButton("Search");
        JButton updateButton = new JButton("Update");
        JButton deleteButton = new JButton("Delete");
        JButton clearButton = new JButton("Clear");

        JPanel buttons = new JPanel(new FlowLayout());
        buttons.add(addButton);
        buttons.add(viewButton);
        buttons.add(searchButton);
        buttons.add(updateButton);
        buttons.add(deleteButton);
        buttons.add(clearButton);

        JPanel topPanel = new JPanel(new BorderLayout());
        topPanel.add(form, BorderLayout.CENTER);
        topPanel.add(buttons, BorderLayout.SOUTH);

        add(topPanel, BorderLayout.CENTER);

        model = new DefaultTableModel(
            new String[]{
                "Issue ID", "Book ID", "Student Name",
                "Issue Date", "Return Date"
            }, 0
        );

        table = new JTable(model);
        add(new JScrollPane(table), BorderLayout.SOUTH);

        connectDatabase();

        addButton.addActionListener(e -> addRecord());
        viewButton.addActionListener(e -> viewRecords());
        searchButton.addActionListener(e -> searchRecord());
        updateButton.addActionListener(e -> updateRecord());
        deleteButton.addActionListener(e -> deleteRecord());
        clearButton.addActionListener(e -> clearFields());

        table.getSelectionModel().addListSelectionListener(e -> {
            if (!e.getValueIsAdjusting()
                    && table.getSelectedRow() >= 0) {
                int row = table.getSelectedRow();

```

TelemedicineBilling.java | SmartLogistics.java | Untitled-2 | JDBCSelect.java | BookIssueTrac

BookIssueTrackingGUI.java > ...

```java
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.sql.*;

public class BookIssueTrackingGUI extends JFrame {

    JTextField issueIdField, bookIdField, studentField;
    JTextField issueDateField, returnDateField;

    JTable table;
    DefaultTableModel model;
    Connection con;

    public BookIssueTrackingGUI() {
        setTitle("Book Issue Tracking System");
        setSize(900, 600);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout(10, 10));

        JLabel heading = new JLabel(
            "BOOK ISSUE TRACKING SYSTEM", JLabel.CENTER
        );
        heading.setFont(new Font("Arial", Font.BOLD, 22));
        heading.setBorder(BorderFactory.createEmptyBorder(15, 5, 10, 5));
        add(heading, BorderLayout.NORTH);

        issueIdField = new JTextField();
        bookIdField = new JTextField();
        studentField = new JTextField();
        issueDateField = new JTextField();
        returnDateField = new JTextField();

        JPanel form = new JPanel(new GridLayout(5, 2, 10, 10));
        form.setBorder(BorderFactory.createEmptyBorder(10, 15, 10, 15));

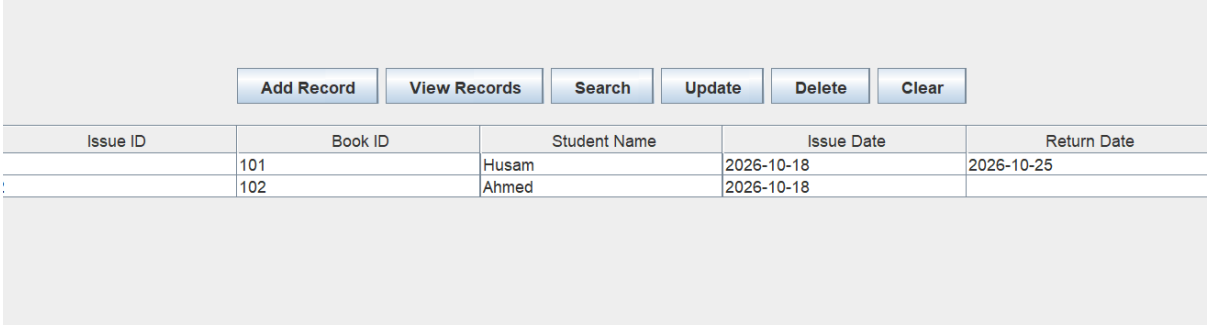
        form.add(new JLabel("Issue ID:"));
        form.add(issueIdField);

        form.add(new JLabel("Book ID:"));
        form.add(bookIdField);

        form.add(new JLabel("Student Name:"));
        form.add(studentField);

        form.add(new JLabel("Issue Date (YYYY-MM-DD):"));
```

Output



Conclusion

The JDBC with GUI application successfully integrates Java Swing with a database using JDBC. It provides a user-friendly graphical interface to perform database

operations efficiently, demonstrating the use of Java GUI components and database connectivity.